

PORTFOLIO CASE STUDY

Pharmacy Inventory Monitoring System

An Automated Daily Stock & Expiry Alert System Built in n8n

• n8n • PostgreSQL • SQL • JavaScript • Gmail SMTP • HTML Email

1. Project Overview

A backend automation workflow that checks a pharmacy's medicine inventory, detects items that are low in stock or expiring within 30 days, builds an HTML report, emails it automatically, and logs the alert. Built and tested in n8n against a real PostgreSQL database.

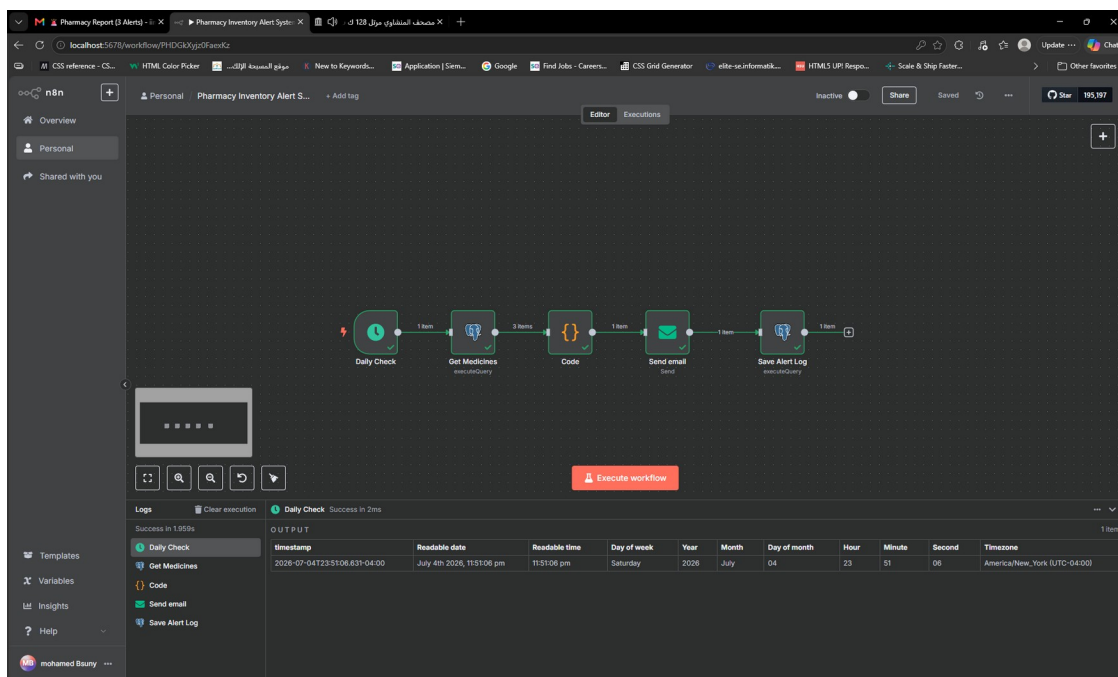


Figure 1 — Full workflow: Daily Check → Get Medicines → Code → Send Email → Save Alert Log.

2. Problem Statement

Pharmacies commonly rely on staff remembering to manually check stock levels and expiry dates. In practice this gets skipped, which leads to running out of important medicines, expired stock sitting on shelves, and staff time spent on a repetitive manual check.

3. Solution

- 1. **Trigger** – A scheduled trigger ("Daily Check") starts the workflow.
- 2. **Database read** – A PostgreSQL query reads the medicines table and classifies each item as LOW_STOCK or EXPIRING_SOON using a SQL CASE expression.

- **3. Report generation** – A Code node loops through the results and builds an HTML report with two tables: low-stock medicines and medicines expiring soon.
- **4. Email notification** – An email with the HTML report and a subject line that reflects the live alert count (e.g. "Pharmacy Report (3 Alerts)") is sent via Gmail SMTP.
- **5. Alert logging** – A final node writes a log entry to PostgreSQL confirming the alert email was sent.

Note: The workflow is now set to Active in n8n (confirmed in Figure 1a), so it runs on its own schedule without manual intervention.

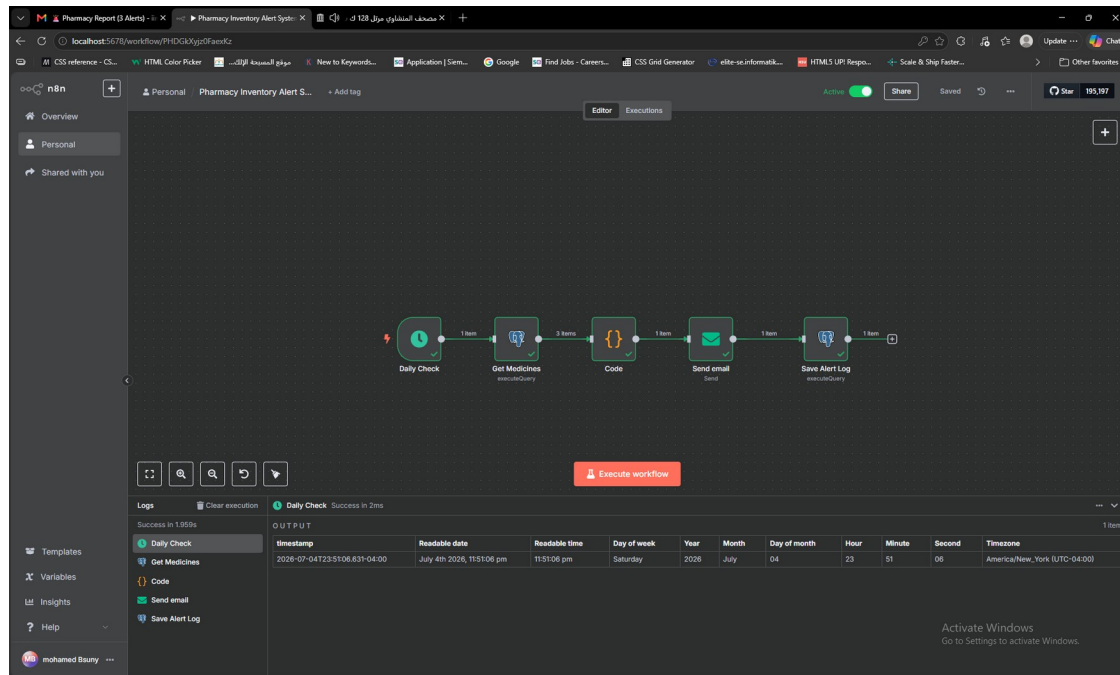


Figure 1a — Workflow switched to Active, now running unattended on its own schedule.

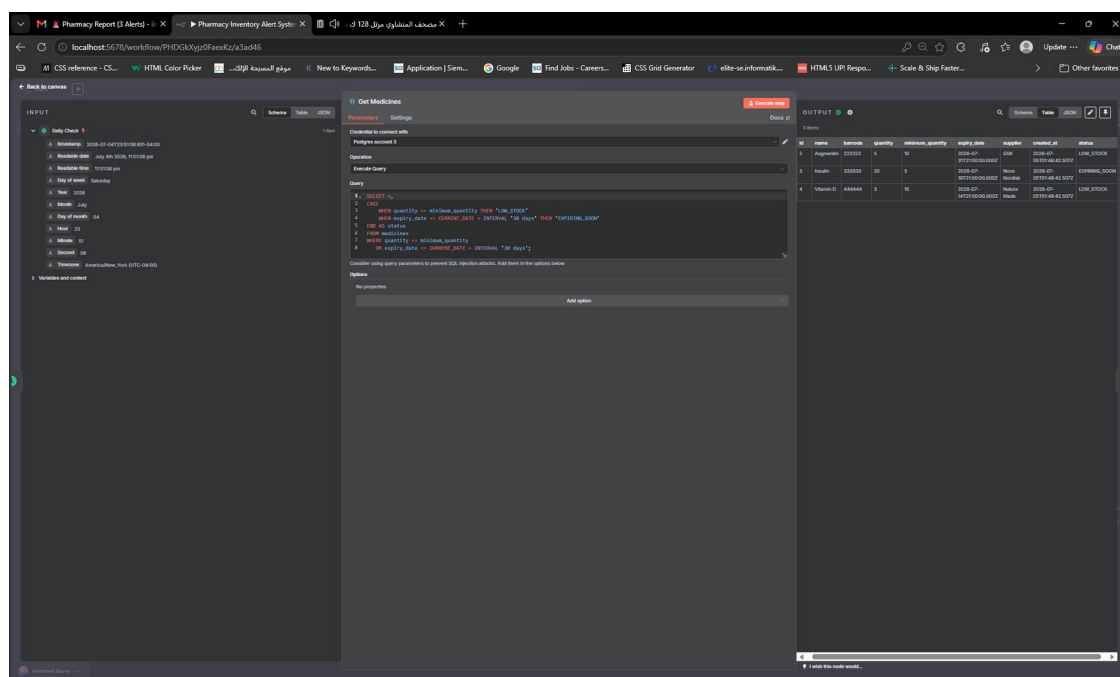


Figure 2 — SQL query classifying medicines as LOW_STOCK or EXPIRING_SOON directly in PostgreSQL.

4. Architecture

Daily Check (Schedule Trigger) → Get Medicines (PostgreSQL SELECT + CASE) → Code (build HTML report) → Send Email (Gmail SMTP) → Save Alert Log (PostgreSQL INSERT)

Core Components

Layer	Technology	Role
Orchestration	n8n (local instance)	Runs the 5-node sequence; currently Inactive (manually triggered for testing)
Data Layer	PostgreSQL + SQL	Stores medicines; a CASE expression classifies stock/expiry status at query time
Logic	JavaScript (Code node)	Splits results into low-stock vs. expiring, builds the HTML report string
Notifications	Gmail SMTP	Sends the HTML report by email with a dynamic subject line

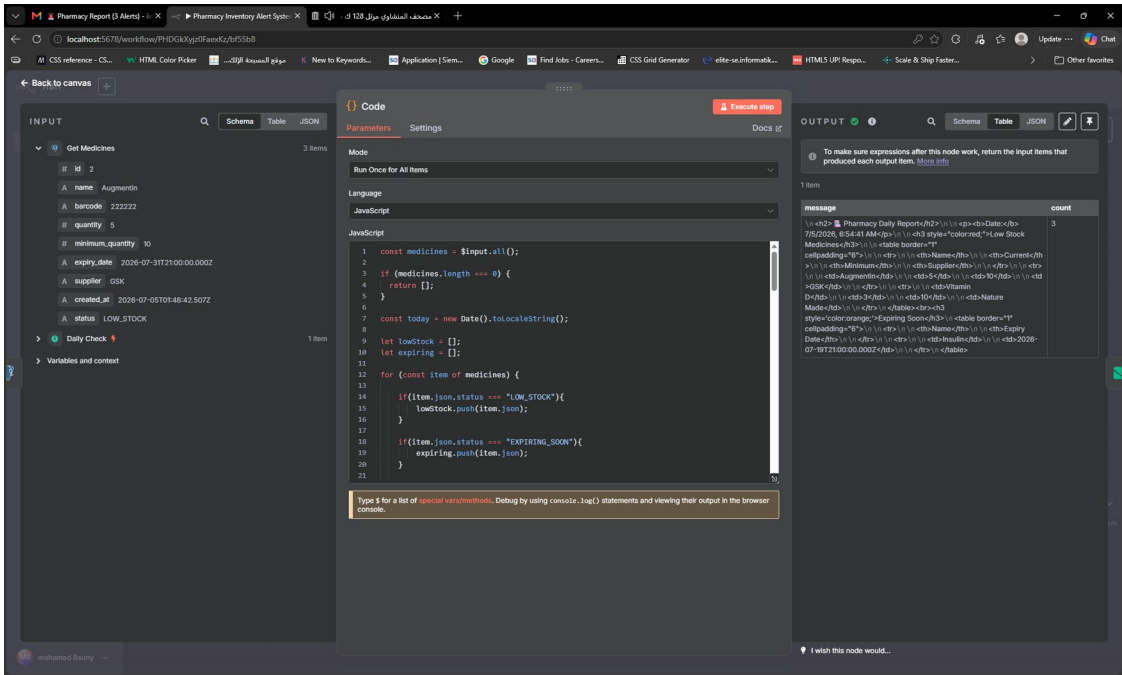


Figure 3 — Code node building the HTML report from the classified medicine data.

5. Database Design

One core table was used directly by the workflow:

medicines

Column	Type	Notes
id	integer (PK)	Identifier

Column	Type	Notes
name	text	Medicine name
barcode	text	Product barcode
quantity	integer	Current stock count
minimum_quantity	integer	Reorder threshold
expiry_date	timestamp	Expiry date used for the 30-day check
supplier	text	Supplier name
created_at	timestamp	Row creation time

Note: "status" (LOW_STOCK / EXPIRING_SOON) is not a stored column — it's computed on the fly by the SQL CASE expression each time the query runs (see Figure 2), which avoids storing stale/duplicated state.

alerts (confirmed via live query)

Column	Type	Notes
id	integer (PK)	Identifier
medicine_id	integer	Not currently populated (NULL in the confirmed test row)
alert_type	varchar(50)	Currently logs a generic "EMAIL_SENT" event, not per-medicine alert type
sent_at	timestamp	When the alert email was sent

Note: The confirmed data shows one summary row per email sent (e.g. alert_type = "EMAIL_SENT", medicine_id = NULL), not one row per flagged medicine. This is a simpler, working design — just worth describing accurately rather than as per-medicine audit logging.

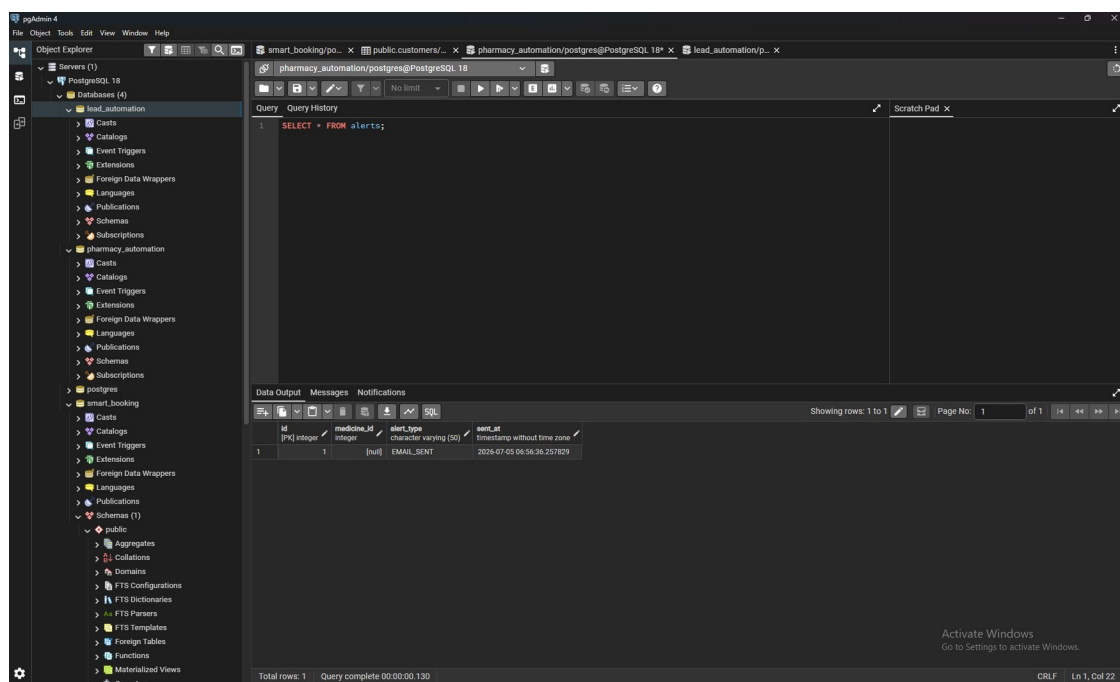


Figure 6 — Confirmed alerts table content: one summary log row per email sent.

6. Features (Implemented)

- **Daily check trigger** – A scheduled trigger node starts the check (needs to be set Active to run unattended).
- **Low-stock & expiry detection** – A single SQL query flags medicines that are low in stock or expiring within 30 days.
- **Automated report generation** – Medicine data is classified and formatted into a two-section HTML report by a Code node.
- **Dynamic email notification** – The report is emailed automatically, with a subject line that reflects the live alert count.
- **Alert logging** – A summary log row (confirmed via live query) is written to PostgreSQL each time an alert email is sent.
- **Tested end-to-end** – Verified end-to-end: SQL classification → HTML report → email received with matching data → log row confirmed in the database.
- **Live scheduled execution** – Confirmed running on its own schedule (workflow status: Active) rather than manually triggered.

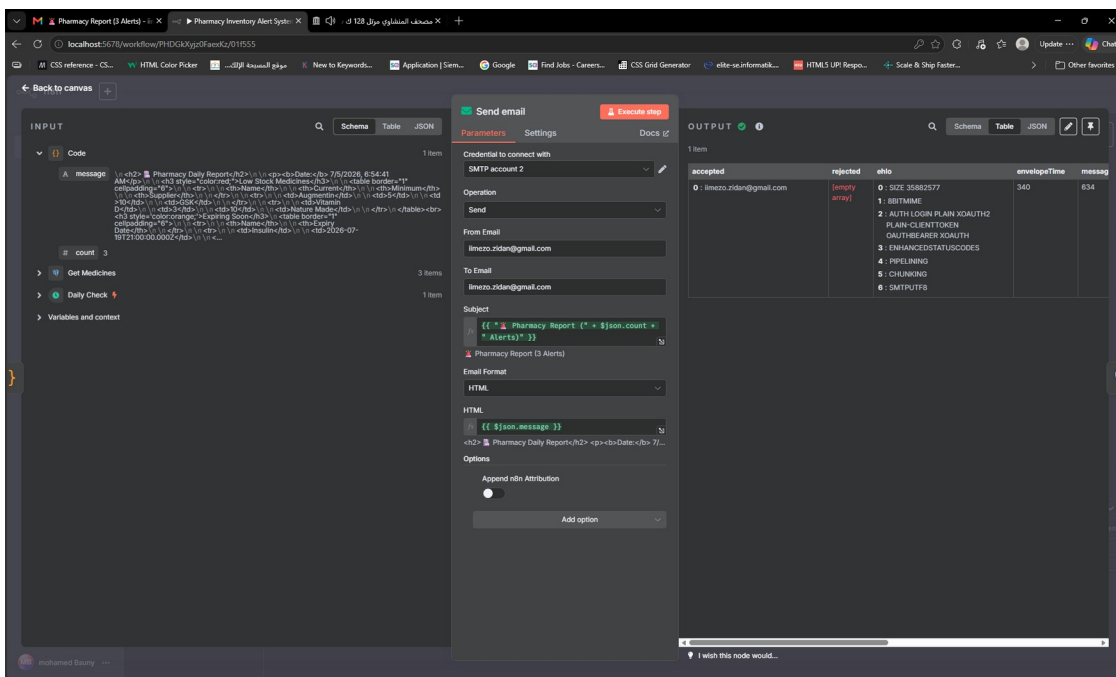


Figure 4 — Send Email node: dynamic subject line and HTML body generated from the report data.

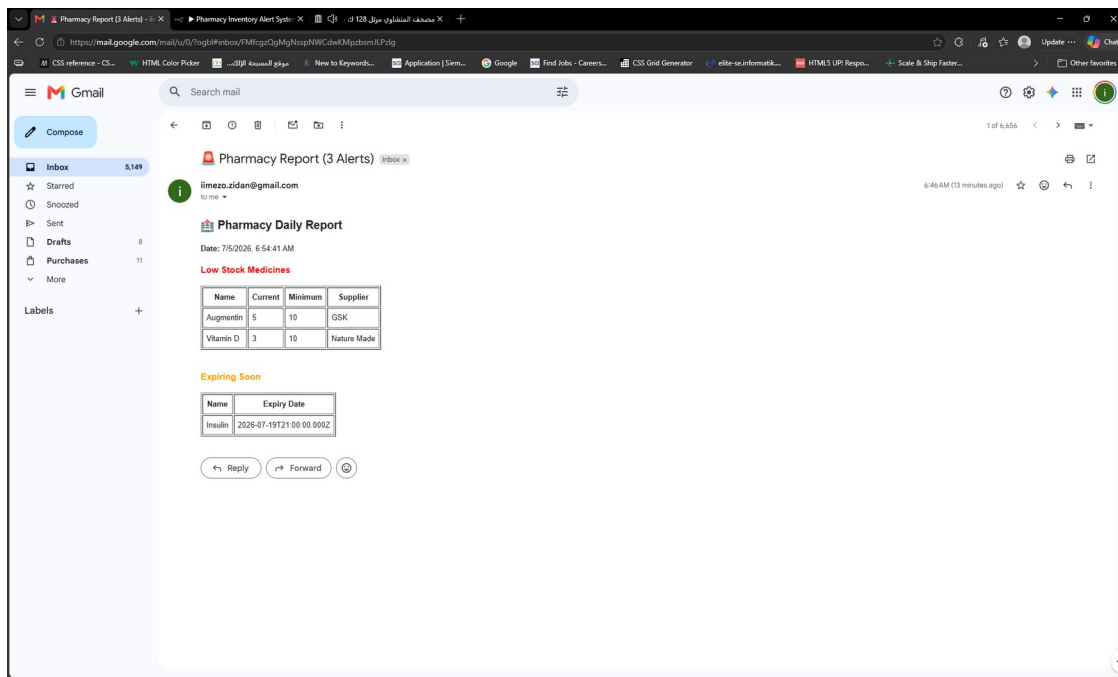


Figure 5 — The actual email received, showing the low-stock and expiring-soon tables.

7. SQL Logic

The core filtering logic runs entirely inside PostgreSQL:

```
WHERE quantity <= minimum_quantity OR expiry_date <= CURRENT_DATE + INTERVAL '30 days'
```

Note: n8n itself flags that this query should use parameterized values to guard against SQL injection. It's a low-risk static query today since no external/user input feeds into it, but this is worth fixing before connecting the workflow to any user-facing input.

One nuance worth noting: the CASE expression assigns LOW_STOCK before checking expiry, so a medicine that is both low in stock and expiring soon will currently only be tagged LOW_STOCK. A medicine's expiry risk could be masked if it's already flagged for stock — worth revisiting if both conditions need independent visibility.

8. Business Value

- **Time saved** – Removes the need for staff to manually check stock levels and expiry dates every day.
- **Fewer shortages** – Low-stock medicines are flagged automatically instead of being discovered when they run out.
- **Reduced expired-stock losses** – Expiring medicines are surfaced 30 days in advance, giving time to sell, return, or discount them before they're wasted.

9. Known Limitations & Future Improvements

- **[Future] Per-medicine alert logging** – The alerts table logs one generic "EMAIL_SENT" row per run with medicine_id left NULL, rather than one row per flagged medicine with its specific alert type — worth adding for a proper audit trail.
- **[Future] Production email config** – From/To email are currently both the developer's own test address; a production version needs the pharmacist's real email, configurable per pharmacy.
- **[Future] Parameterized SQL queries** – Query should use parameters instead of a static string, per n8n's own warning.
- **[Future] Independent low-stock/expiry flags** – A medicine that is both low-stock and expiring is only tagged with one status today.
- **[Future] Additional channels** – WhatsApp/Telegram notifications alongside email.
- **[Future] Extended features** – PDF report export, a simple dashboard, multi-branch support, and automatic supplier reordering.

10. Skills Demonstrated

Category	Skills
Backend Automation	n8n scheduled workflow design
Database Engineering	SQL CASE-based classification logic in PostgreSQL
Scripting	JavaScript (Code node) for data transformation and HTML generation
Integration	Gmail SMTP for automated, dynamically-generated email reports

11. Portfolio-Ready Description

Built an automated pharmacy inventory monitoring workflow in n8n that reads stock data from PostgreSQL, flags medicines that are low in stock or expiring within 30 days using SQL, generates a live HTML report, and emails it automatically with alert logging. Tested end-to-end with real data — from SQL classification through to a delivered, accurately-populated email report.